Preference Learning Pipeline for MidiBERT-Piano:

Dataset Generation and Reward Model Fine-Tuning for Symbolic Music

Aditya Kinjawadekar

March 2026

Abstract

Evaluating and aligning generative models for symbolic music remains a significant challenge due to the subjective nature of musical quality and the absence of robust automatic metrics. Reinforcement Learning from Human Feedback (RLHF) offers a promising solution, but requires a pre-trained reward model capable of judging musical coherence. We present a complete, end-to-end preference learning pipeline built on top of MidiBERT-Piano [1]. The framework consists of two primary components: (1) a sophisticated dataset generator that tokenizes MAESTRO v3.0.0 MIDI files into Compound Word (CP) representation and produces n^2 preference pairs via a suite of targeted corruption strategies (simulating structural, harmonic, and rhythmic degradation); and (2) a parameter-efficient fine-tuning (PEFT) script that attaches a scalar reward head to the MidiBERT encoder, training it with a Bradley–Terry preference loss using Low-Rank Adaptation (LoRA). This expansive document provides the necessary theoretical context, architectural motivations, and implementation details for the pipeline.

1 Introduction and Motivation

Generative models for symbolic music, typically based on Transformer architectures, have achieved remarkable success in composing realistic piano performances. However, these models are traditionally trained via maximum likelihood estimation (MLE) on datasets of human performances. MLE training forces the model to mimic the training data distribution but fails to distinguish between high-quality compositions and mediocre ones, nor does it explicitly discourage structurally incoherent generations.

In natural language processing, Reinforcement Learning from Human Feedback (RLHF) has bridged this gap by aligning language models with human preferences. The RLHF pipeline relies on a **Reward Model (RM)** that assigns a scalar quality score to a generated sequence. To train an RM for music, we require a dataset of preference pairs: instances where one musical excerpt is explicitly preferred over another.

Collecting human preference data for music is notoriously expensive and time-consuming. To bootstrap the RM, we employ a **synthetic preference dataset** strategy. By taking high-quality, human-performed classical music (the MAESTRO dataset) and applying controlled, musically-destructive corruption algorithms, we can automatically generate pairs where the original, uncorrupted version is unconditionally preferred.

2 Background: Symbolic Music Representation

A fundamental decision in processing symbolic music is the choice of tokenization. Historically, formats like MIDI-like or REMI (Revamped MIDI-derived events) flatten all musical events (Note-On, Note-Off, Velocity, Time-Shift) into a 1D sequence. This flattening breaks the semantic grouping of note attributes, forcing the Transformer to learn that a "Pitch" token is intrinsically linked to the preceding "Duration" token.

2.1 Compound Word (CP) Tokenization

MidiBERT [1] adopts the **Compound Word (CP)** representation [2], drastically reducing sequence length and explicitly grouping note attributes. In CP, a single token is not a single integer, but a **4-tuple** representing four distinct characteristics of a note event.

Let $\mathbf{t}$ be a CP token. It is defined as:

$$\mathbf{t} = \left[\, t_{\text{Bar}},\ t_{\text{Pos}},\ t_{\text{Pitch}},\ t_{\text{Dur}} \,\right] \tag{1}$$

- **Bar (t_{Bar})**: Indicates whether this token begins a new bar (`Bar New`) or continues an existing one (`Bar Continue`). Vocabulary size: 4 (includes padding and masking).
- **Position (t_{Pos})**: Represents the onset time of the note within the current bar, discretized into 16 bins. Vocabulary: 18 (16 fractions + pad + mask).
- **Pitch (t_{Pitch})**: The MIDI pitch value, typically restricted to the 88 piano keys (MIDI notes 22 to 109). Vocabulary: 90 (88 pitches + pad + mask).
- **Duration (t_{Dur})**: The length of the note, quantized into 64 distinct time bins ranging from 60 ticks to 3840 ticks. Vocabulary: 66 (64 bins + pad + mask).

2.2 MidiBERT Embedding Strategy

Because the token is multi-dimensional, MidiBERT embeds each of the four attributes independently into a $d_{\text{attr}} = 256$ dimensional space using separate learned lookup tables. These four embeddings are then concatenated to form a 1024-dimensional vector, which is linearly projected down to the BERT hidden size ($d_{\text{model}} = 768$).

$$\mathbf{h}_{\text{emb}} = W_{\text{proj}} \left[\, E_{\text{Bar}}(t_{\text{Bar}}) \parallel E_{\text{Pos}}(t_{\text{Pos}}) \parallel E_{\text{Pitch}}(t_{\text{Pitch}}) \parallel E_{\text{Dur}}(t_{\text{Dur}}) \,\right] \tag{2}$$

This architecture allows the attention mechanism to directly access and correlate independent musical features simultaneously, rather than waiting for recurrent updates across a 1D sequence.

3 Dataset Generation Pipeline

The first script in our pipeline, `generate_preference_dataset.py`, processes the MAESTRO v3.0.0 dataset [4]. MAESTRO contains 200 hours of virtuosic piano performances.

3.1 Standardizing and Tokenizing

We utilize the preprocessing tools native to the MidiBERT repository to ensure exact compatibility with the pre-trained weights.

1. **Parsing**: The `miditoolkit` library extracts Note and Tempo objects.
2. **Quantization**: Musical time is Continuous. We quantize it to a discrete grid with a resolution of 480 ticks per quarter note. Note onsets are snapped to the nearest 120-tick boundary.
3. **Grouping and Conversion**: Notes are grouped chronologically by Bar. The tuples are mapped to integer IDs using the pre-computed `CP.pkl` dictionary (or extracted from the checkpoint state dict).
4. **Chunking**: Since BERT models have quadratic attention complexity, sequence length is bounded. We slice the CP sequences into non-overlapping chunks of $L = 512$ tokens. Sequences shorter than 512 are padded with a designated 4-tuple `[Bar<PAD>, Pos<PAD>, Pitch<PAD>, Dur<PAD>]`.

3.2 Targeted Corruption Strategies

To construct preference pairs, we must simulate the types of objective errors a generative model might make. We implemented five corruption strategies that operate directly on the discrete CP token dimension arrays.

Table 1: Detailed definitions of the CP token corruption strategies used to generate negative preference examples.

Strategy	Musical Motivation & Implementation
`shuffle_bars`	**Tests structural coherence**. Generative models often produce locally coherent music that wanders globally. We identify bar boundaries (where $t_{\text{Bar}} =$ `Bar New`), split the sequence into bar-level blocks, and randomly shuffle these blocks. This destroys macroscopic structure while preserving local within-bar harmony.
`randomize_pitch`	**Tests harmonic mapping**. We replace the t_{Pitch} integer for every non-padding token with a random sample from a uniform distribution $\mathcal{U}(0, 87)$. The rhythm remains pristine, but the resultant audio sounds highly dissonant.
`randomize_position`	**Tests rhythmic jitter**. Models sometimes struggle with micro-timing. We replace the t_{Pos} integer (the sub-bar fraction) with uniform noise. Notes occur at the correct bar, and play the correct pitch for the correct duration, but their onsets are chaotic.
`note_dropout`	**Tests sparsity and continuity**. We randomly drop 40% of the tokens. To restore the tensor to length 512, we append padding tokens at the end. This simulates a model "forgetting" to play notes, leaving awkward silences.
`randomize_duration`	**Tests articulation loss**. We replace t_{Dur} with uniform noise. This alters the musical phrasing (staccato vs legato) without changing the melody or rhythm.

3.3 Generating $\mathcal{O}(n^2)$ Pairs

If we tokenize n unique MIDI segments, producing n original token arrays and n corrupted variants, we generate the full Cartesian product to maximize dataset size and contrastive diversity:

$$\mathcal{D} = \{ \, (\mathbf{x}_i^{\text{orig}}, \, \mathbf{x}_j^{\text{corrupt}}, \, y = 1) \mid i, j \in \{1, \ldots, n\} \, \} \tag{3}$$

By pairing original file i with the corruption of file j, the RM must learn that *any* clean, human-composed piece is preferable to *any* structurally broken piece, preventing the model from merely memorizing specific songs.

4 Reward Model Architecture and Fine-Tuning

The second script, `finetune_reward_model.py`, transforms the pre-trained, masked-language-model MidiBERT into a discriminative Reward Model.

4.1 Architecture

The RM processes an input sequence $\mathbf{X} \in \mathbb{Z}^{512 \times 4}$ and outputs a single scalar value $r \in \mathbb{R}$.

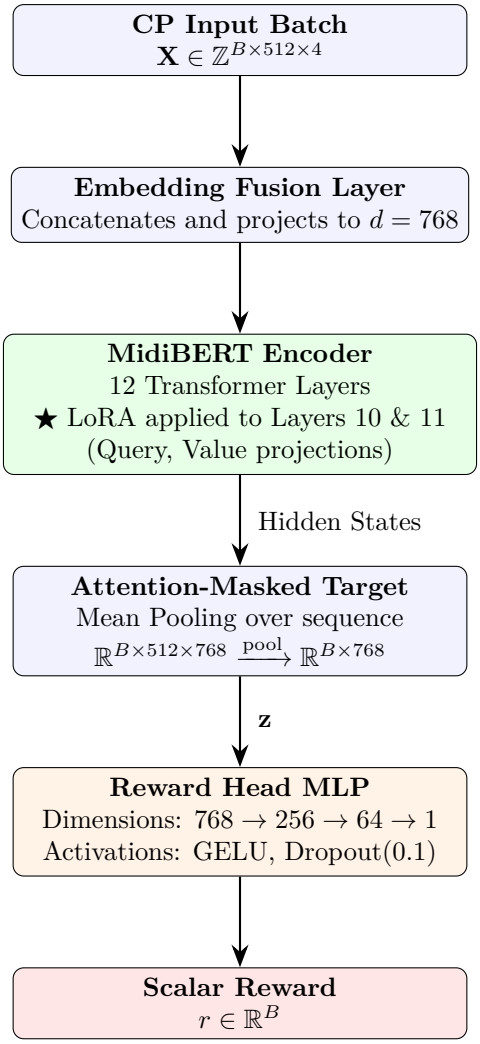

Figure 1: The end-to-end Reward Model architecture. The base MidiBERT model is frozen, and only the LoRA adapters (green) and the newly initialized Reward Head (orange) are updated during gradient descent.

Due to CP padding, sequences vary in effective length. Simple mean pooling would incorrectly include padding tokens in the aggregate state. We use **Attention-Masked Mean Pooling**:

$$\mathbf{z} = \text{LayerNorm}\left(\frac{\sum_{i=1}^{L} m_i \cdot \mathbf{h}_i}{\sum_{i=1}^{L} m_i}\right) \quad \text{where} \quad m_i = \begin{cases} 1 & \text{if } t_{\text{Bar},i} \neq \texttt{Bar<PAD>} \\ 0 & \text{otherwise} \end{cases} \tag{4}$$

4.2 Parameter-Efficient Fine-Tuning (PEFT)

Training a 85M parameter BERT model from scratch on small datasets is prone to catastrophic forgetting and overfitting. Rather than full fine-tuning, we utilize **Low-Rank Adaptation (LoRA)** [3].

LoRA freezes the pre-trained weights $W_0 \in \mathbb{R}^{d\times k}$ and injects trainable rank decomposition matrices into each layer:

$$W_{\text{forward}} = W_0 + \Delta W = W_0 + \frac{\alpha}{r_{rank}} BA \tag{5}$$

where $B \in \mathbb{R}^{d\times r_{rank}}$ and $A \in \mathbb{R}^{r_{rank}\times k}$.

We target the self-attention **Query** and **Value** matrices of the **last 2 layers** of the 12-layer encoder. With $r_{rank} = 8$ and $\alpha = 16$, the trainable parameter count drops to approximately 0.5% of the full model size, significantly accelerating training speed and reducing GPU VRAM requirements.

4.3 Bradley-Terry Preference Optimization

The reward model does not output a probability; it outputs a raw, uncalibrated real number. To train it, we treat preference learning as a ranking optimization problem using the Bradley-Terry ranking algorithm [5].

Given a preference dataset $\mathcal{D}$ consisting of pairs $(\mathbf{x}^w, \mathbf{x}^l)$ where $\mathbf{x}^w$ is the "winning" (preferred) sequence, the probability that a human prefers $\mathbf{x}^w$ over $\mathbf{x}^l$ is modeled by the sigmoid of the difference in their rewards:

$$P(\mathbf{x}^w \succ \mathbf{x}^l) = \sigma\big(r(\mathbf{x}^w) - r(\mathbf{x}^l)\big) = \frac{1}{1 + \exp\big(-(r(\mathbf{x}^w) - r(\mathbf{x}^l))\big)} \tag{6}$$

The network is trained to minimize the negative log-likelihood of this probability:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(\mathbf{x}^w, \mathbf{x}^l) \sim \mathcal{D}} \Big[\log \sigma\big(r(\mathbf{x}^w) - r(\mathbf{x}^l)\big)\Big] \tag{7}$$

This loss forces the RM to increase the scalar gap between clean and corrupted sequences.

5 Execution and Further Applications

The implementation leverages the `uv` package manager for deterministic virtual environments, tracking dependencies like `peft`, `transformers`, and `miditoolkit`.

The output of `finetune_reward_model.py` includes a `best_reward_model.pt` checkpoint containing the PEFT adapter weights and the Reward Head values.

Downstream usage: This RLHF Reward Model is intended to evaluate sequences generated by a policy model (e.g., an autoregressive GPT trained on MidiBERT CP tokens). By computing $r(\mathbf{x}_{\text{gen}})$, engineers can automatically filter generative outputs, or mathematically integrate the RM directly into a Proximal Policy Optimization (PPO) loop to fine-tune the generator to avoid structural incoherence.

6 Model Dimensions Summary

The architectural dimensions used in the standard configuration of our MidiBERT-Piano preference learning pipeline are summarized as follows:

- **Input Array:** $B \times 512 \times 4$ (Batch size $\times$ Sequence Length $\times$ Compound Word tuple).
- **Token Embeddings:** Each of the 4 token attributes is embedded into $d_{\text{attr}} = 256$.
- **Concatenated Embeddings:** $B \times 512 \times 1024$.
- **BERT Input Projection:** $1024 \rightarrow d_{\text{model}} = 768$.
- **MidiBERT Transformer:** 12 layers, 12 attention heads, hidden dimension $d_{\text{model}} = 768$.
- **LoRA Sub-matrices:** Rank $r = 8$, applied to the Query and Value projections of the final 2 layers. Learnable parameters $\Delta W = BA$, where $B \in \mathbb{R}^{768 \times 8}$ and $A \in \mathbb{R}^{8 \times 768}$.
- **Mean Pooled State:** $B \times 768$ (extracted using the padding-mask).

- **Reward Head MLP Layers:** $768 \to 256 \to 64 \to 1$.
- **Final Reward Output:** $B \times 1$ scalar per sequence.

References

[1] Y.-H. Chou, I.-C. Chen, C.-J. Chang, J. Ching, and Y.-H. Yang, "MidiBERT-Piano: Large-scale pre-training for symbolic music understanding," *arXiv preprint arXiv:2107.05223*, 2021.

[2] W.-N. Hsiao, J.-Y. Liu, Y.-C. Yeh, and Y.-H. Yang, "Compound Word Transformer: Learning to Compose Full-Song Music over Dynamic Directed Hypergraphs," *AAAI*, 2021.

[3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," *ICLR*, 2022.

[4] C. Hawthorne, et al., "Enabling factorized piano music modeling and generation with the MAESTRO dataset," *ICLR*, 2019.

[5] R. A. Bradley and M. E. Terry, "Rank analysis of incomplete block designs: I. The method of paired comparisons," *Biometrika*, vol. 39, no. 3/4, pp. 324–345, 1952.